01 — Overview

The problem

You're given:

- **`candidates.jsonl`** — 100,000 anonymised candidate profiles, one JSON object per line. Each has a profile (headline, summary, title, location, years of experience), a career history (roles, companies, durations, descriptions), skills (with endorsements and self-rated proficiency), behavioural signals (activity, recruiter responses, assessment scores), and verification flags.
- **A job description (JD)** — for the hackathon, a "Senior AI Engineer — Founding Team" role.

You must produce **`submission.csv`** — the **top 100** candidates ranked best-fit first, each with a `score` and a one-sentence `reasoning`.

The hard constraints (these shape every design decision)

1. **No network and no GPU during ranking.**
2. **CPU only, ≤ 16 GB RAM, under 5 minutes** for the ranking step.
3. **Exactly 100 rows**, valid against the organiser's `validate_submission.py`.
4. The dataset hides **~80 "honeypot" profiles** — deliberately impossible resumes (e.g. 40 years of experience at age 25) that you must detect and exclude.
5. The official score is a weighted blend of ranking-quality metrics:

   `score = 0.50·NDCG@10 + 0.30·NDCG@50 + 0.15·MAP + 0.05·P@10`

   (Don't worry if those are unfamiliar — they're all defined in 06_glossary.md. In short: did you put the right people near the top?)

What the system does (the whole thing)

The project grew in four layers. Each one is useful on its own and degrades gracefully if its dependencies aren't installed.

1. **The static ranker (the graded core).** Reads profiles, builds 40 numeric features per candidate, retrieves the most relevant ones with a hybrid keyword + semantic search, scores them with a machine-learning model, removes honeypots and unqualified candidates, and writes the ranked top-100.
2. **The accuracy layer.** Three techniques that sharpen the *top* of the list, where the score is decided: graded "pseudo-labels" feeding a learning-to-rank model (LambdaMART), a cross-encoder **reranker**, and a pairwise **Bradley-Terry tournament** (our own differentiator). Plus an evaluation harness that measures the exact contest metric so changes are proven, not guessed.
3. **The production service.** A FastAPI application with a real database (Postgres/SQLite), Redis caching, a job queue, authentication (JWT + API keys) with role-based access, Prometheus metrics, structured logging, distributed tracing, a circuit breaker, rate limiting, Docker packaging, CI, and a React recruiter UI.
4. **The live signal ingestion layer.** A passive career-intelligence agent that pulls a developer's real GitHub activity, turns each piece of work into evidence, scores how much that evidence is worth (with recency decay), stores it in a SHA-256-anchored "proof-of-work" knowledge graph, and lets a recruiter query it in plain English. It composes with the static ranker through a small additive "evidence bonus" — it never overrides the graded submission.
5. **The performance layer.** Speed levers that make the offline phase cheaper without weakening any feature: GPU auto-detection, an ONNX/int8 toggle, leaner model training, a focused rerank text, and a content-hash cache for re-runs.

The full technology stack

Core ranking engine (pure-Python-capable)

Concern	Technology	Notes
Numerics / data	**NumPy**, **pandas**	The feature matrix is a pandas DataFrame
Keyword search	**rank-bm25** (Okapi BM25)	Falls back to a hand-written streaming inverted index
Semantic search	**sentence-transformers** (MiniLM, 384-dim)	Falls back to a NumPy feature-hashing embedder
Fusion	**Reciprocal Rank Fusion** (custom)	Combines keyword + semantic ranks
ML model	**XGBoost** (`XGBRanker`, LambdaMART)	Falls back to LightGBM → scikit-learn → a formula
Reranker	**sentence-transformers** `CrossEncoder`	Falls back to a lexical BM25-style scorer
Tournament	**NumPy** (Bradley-Terry MLE)	`scipy` listed but the core is pure NumPy
Model I/O	**joblib**	Saves/loads the trained model artifact
LLM (optional, offline)	**google-generativeai** (Gemini)	JD parsing + label grading; heuristic fallback

Production service

Concern	Technology
Web framework	**FastAPI** + **uvicorn** / **gunicorn**
Streaming progress	**sse-starlette** (Server-Sent Events)
Database (async)	**SQLAlchemy 2.0** + **asyncpg** (Postgres) / **aiosqlite** (dev)
Migrations	**Alembic**
Cache + rate limit	**Redis** (with an in-memory fallback)
Auth	**PyJWT** (JWT) + **bcrypt** (passwords) + per-user API keys
Validation	**Pydantic v2** + **pydantic-settings**
Metrics	**prometheus-client**
Logging	**structlog** (JSON to stdout)
Tracing	**OpenTelemetry** (optional, exports to Jaeger)
Background work	**Celery** (eager fallback = runs inline)
Knowledge graph	**Neo4j** (with an in-memory fallback)
Resume parsing	**pdfplumber** / **python-docx** (optional)

Frontend

Concern	Technology
Framework	**React 18** + **TypeScript**
Build tool	**Vite**
Styling	**Tailwind CSS**
Data fetching	**TanStack React Query**
Routing	**react-router-dom**
Charts	**Recharts**
Icons	**lucide-react**

Infrastructure & quality

Concern	Technology
Containers	**Docker** multi-stage + **docker-compose** (13 services)
Reverse proxy / LB	**nginx** (the `gateway` service)
Dashboards	**Grafana** + **Prometheus**
Tracing UI	**Jaeger**
CI	**GitHub Actions**
Lint / format	**Ruff**

Concern	Technology
Tests	**pytest** + **pytest-asyncio** (68 tests)

Repository map (top level)

```
provenancerank/
├── precompute.py          # PHASE 1 (offline): build all cached artifacts
├── rank.py                # PHASE 2 (online): read cache -> submission.csv
├── validate_submission.py # organiser's official validator (do not edit)
│
├── core/          # config, logging, constants, security, device, cache, errors
├── pipeline/      # the ranking engine: features, retrieval, scoring, honeypots
├── ml/            # model training, prediction, pseudo-labels, reranker, metrics
├── output/        # reasoning generator + submission writer
│
├── api/           # FastAPI app: routers, schemas, deps, middleware, metrics
├── db/            # SQLAlchemy models, repositories, engine, migrations
├── services/      # ranker engine wrapper, job queue, cache, rate limiter
│
├── ingestion/     # GitHub client, resume parser, sync core, Celery tasks
├── intelligence/  # confidence scorer, skill extractor, artifact summariser
├── graph/         # knowledge-graph schema, read/write, natural-language query
│
├── frontend/      # React + TypeScript recruiter UI
├── alembic/       # database migration scripts
├── deploy/        # Prometheus + Grafana provisioning
├── scripts/       # dev helpers (e.g. build a small sample dataset)
├── tests/         # pytest suite (63 tests)
│
├── docker-compose.yml, Dockerfile, .github/workflows/ci.yml
└── docs/          # you are here
```

Every one of these files is described in 04_file_by_file.md. The next doc, 02_architecture.md, explains how they fit together.